

# **Module 7**

## Real-World Applications

# Learning objectives

- Apply UVM to real-world verification scenarios

# Prerequisites

- See module README

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["DMA Verification"]

T2["Protocol Verification (Indus)"]

T1 --> T2

T3["Best Practices and Patterns"]

T2 --> T3

Apply UVM to real-world verification scenarios

# Overview

- This module applies all learned concepts to real-world verification scenarios. You'll work on compl...

# **Design architecture**

# RTL / block architecture

## Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph soc["module7 IP blocks"]

DMA[simple\_dma]

UART[uart]

end

BUS[internal control] --> DMA

Module 7: DUT structure and key interfaces (see common\_dut/rtl/).

# Verification environment architecture

## Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

VIP[verification IP agents] --> ENV[uvm\_env]

ENV --> DMA[(DMA DUT)]

ENV --> UART[(UART DUT)]

ENV --> SB[system scoreboard]

Module 7: UVM layers, agents, and checkers for this phase.

# 1. Real-world DUT blocks

- ``module7/dut/dma/simple_dma.v`` — DMA controller with channel descriptors
- ``module7/dut/protocols/uart.v`` — serial framing for protocol agent labs
- Blocks represent IP-style interfaces students will see in SoC verification

## 2. Verification IP (VIP) architecture

- Reusable agents package protocol knowledge (transactions, sequences, checkers)
- Env composes DMA + UART (or other) agents with shared config and scoreboard fabric
- Best-practices example shows directory layout, naming, and review-friendly TB structure

### 3. Integration view

- Software-visible registers drive DMA; UART provides async serial path
- Scoreboard and coverage span multiple IPs in one regression
- Exercises push toward production-style reuse and documentation

# **Verification & testing methods**

# Testing and closure flow

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

SM[smoke regressions] --> NG[nightly full EXAMPLES]

NG --> REL[release + coverage]

REL --> SHIP[sign-off]

Module 7: how tests, checks, and metrics close verification goals.

# 1. Application-level test methods

- DMA tests: descriptor programming, transfer completion, interrupt/status flags
- UART tests: baud framing, TX/RX loops, error injection in exercises
- VIP tests encapsulate protocol rules so tests focus on scenarios not bit toggling

## 2. Regression and sign-off practices

- ``./scripts/module7.sh`` runs DMA, protocol, VIP, and best-practices demos
- Layered regressions: smoke (short), nightly (full EXAMPLES), release (with coverage goals)
- Waveforms and transaction logs for post-mortem on failures

### 3. Closure

- ``module7.sh --check``; assessment covers VIP reuse, protocols, and integration
- Prepares for Module 8 utilities used in production regressions (CLP, recorders)

# Syllabus topics

# 1. DMA Verification (1/4)

- DMA Controller Overview
- DMA concepts
- DMA controller architecture
- DMA transfer types
- DMA verification challenges
- DMA Testbench Architecture

# 1. DMA Verification (2/4)

- Register interface agent
- Memory interface agent
- DMA monitor
- Scoreboard design
- Coverage model
- DMA Verification Scenarios

# 1. DMA Verification (3/4)

- Simple transfers
- Scatter-gather transfers
- Multiple channel transfers
- Error scenarios
- Performance verification
- DMA Test Implementation

# 1. DMA Verification (4/4)

- Test scenarios
- Sequence design
- Coverage closure
- Regression testing

## 2. Protocol Verification (Industry Standards) (1/4)

- UART Verification
- UART protocol
- UART agent design
- UART testbench
- UART verification
- SPI Verification

## 2. Protocol Verification (Industry Standards) (2/4)

- SPI protocol
- SPI agent design
- Master-slave coordination
- SPI testbench
- I2C Verification
- I2C protocol

## 2. Protocol Verification (Industry Standards) (3/4)

- I2C agent design
- Multi-master scenarios
- I2C testbench
- AXI Verification
- AXI protocol details
- AXI agent implementation

## 2. Protocol Verification (Industry Standards) (4/4)

- AXI testbench
- AXI compliance

# 3. Best Practices and Patterns (1/4)

- Code Organization
- Project structure
- File organization
- Naming conventions
- Documentation standards
- Reusability Patterns

### 3. Best Practices and Patterns (2/4)

- Component reuse
- Sequence reuse
- Environment reuse
- VIP (Verification IP) creation
- Documentation
- Code documentation

### 3. Best Practices and Patterns (3/4)

- Test documentation
- User guides
- API documentation
- Maintenance
- Code maintenance
- Test maintenance

### 3. Best Practices and Patterns (4/4)

- Version management
- Change management

## 4. Advanced Topics (1/4)

- Performance Optimization
- Testbench optimization
- Simulation speed
- Memory optimization
- CPU utilization
- Coverage Closure

## 4. Advanced Topics (2/4)

- Coverage strategies
- Coverage analysis
- Coverage improvement
- Coverage metrics
- Regression Testing
- Regression strategies

## 4. Advanced Topics (3/4)

- Test selection
- Test execution
- Result analysis
- Continuous Integration
- CI/CD setup
- Automated testing

## 4. Advanced Topics (4/4)

- Result reporting
- Notification systems

# 5. Verification IP (VIP) Development (1/3)

- VIP Overview
- What is VIP?
- VIP components
- VIP structure
- VIP benefits
- VIP Development

## 5. Verification IP (VIP) Development (2/3)

- VIP design
- VIP implementation
- VIP testing
- VIP documentation
- VIP Integration
- VIP integration

## 5. Verification IP (VIP) Development (3/3)

- VIP configuration
- VIP usage
- VIP maintenance

## 6. System-Level Verification (1/2)

- System Verification
- System architecture
- System testbench
- System scenarios
- System verification
- SoC Verification

## 6. System-Level Verification (2/2)

- SoC architecture
- SoC testbench
- SoC scenarios
- SoC verification

# 7. Advanced Debugging (1/2)

- Complex Debugging
- Multi-component debugging
- Transaction flow debugging
- Timing debugging
- Configuration debugging
- Debugging Tools

## 7. Advanced Debugging (2/2)

- Waveform tools
- Log analysis tools
- Coverage tools
- Performance tools

## 8. Test Planning and Strategy (1/2)

- Test Planning
- Test strategy
- Test scenarios
- Test coverage
- Test execution plan
- Verification Strategy

## 8. Test Planning and Strategy (2/2)

- Verification approach
- Verification metrics
- Verification closure
- Sign-off criteria

## 9. Industry Patterns (1/2)

- Common Patterns
- Industry patterns
- Pattern libraries
- Pattern reuse
- Pattern best practices
- Design Patterns

## 9. Industry Patterns (2/2)

- Verification patterns
- Architecture patterns
- Implementation patterns
- Testing patterns

# 10. Project: Build Your Own VIP (1/3)

- Project Requirements
- Choose protocol
- Design VIP
- Implement VIP
- Test VIP
- Document VIP

# 10. Project: Build Your Own VIP (2/3)

- VIP Components
- Complete agent
- Protocol checker
- Coverage model
- Scoreboard
- Register model (if applicable)

# 10. Project: Build Your Own VIP (3/3)

- Documentation
- Test suite

# Hands-on examples

# Module 7 self-check

```
$ ./scripts/module7.sh --check
```

Terminal: module7\_check

```
[[0;36m=====0m
[[0;36mModule 7: Real-World Applications0m
[[0;36m=====0m
```

Usage: ./scripts/module7.sh [OPTIONS]

Module 7: Real-World Applications  
This script runs examples and tests for Module 7.

## OPTIONS:

### Examples:

|                  |                               |
|------------------|-------------------------------|
| --dma            | Run DMA verification examples |
| --uart           | Run UART protocol examples    |
| --spi            | Run SPI protocol examples     |
| --i2c            | Run I2C protocol examples     |
| --vip            | Run VIP development examples  |
| --best-practices | Run best practices examples   |
| --all-examples   | Run all examples (default)    |
| --skip-examples  | Skip all examples             |

### Tests:

|                |                  |
|----------------|------------------|
| --pyuvvm-tests | Run pyuvvm tests |
|----------------|------------------|

### Environment:

|                 |                                                |
|-----------------|------------------------------------------------|
| --venv DIR      | Virtual environment directory (default: .venv) |
| --no-venv       | Don't use virtual environment                  |
| --sim SIMULATOR | Simulator to use (default: verilator)          |

### Other:

|            |                        |
|------------|------------------------|
| --help, -h | Show this help message |
|------------|------------------------|

## EXAMPLES:

```
# Run all examples
./scripts/module7.sh
```

# Demo: DMA Verification

```
$ .venv/bin/python module7/examples/dma/dma_example.py
```

Terminal: ex01\_dma

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/dma/dma\_  
from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Demo: UART Protocol

```
$ .venv/bin/python module7/examples/protocols/uart_example.py
```

Terminal: ex02\_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/protocol  
from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Demo: SPI Protocol

```
$ .venv/bin/python module7/examples/protocols/spi_example.py
```

Terminal: ex03\_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/protocol  
from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Demo: I2C Protocol

```
$ .venv/bin/python module7/examples/protocols/i2c_example.py
```

Terminal: ex04\_protocols

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/protocol

from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Demo: VIP Development

```
$ .venv/bin/python module7/examples/vip/vip_example.py
```

Terminal: ex05\_vip

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/vip/vip\_  
from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Demo: Best Practices

```
$ .venv/bin/python  
    module7/examples/best_practices/best_practices_example.py
```

Terminal: ex06\_best\_practices

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn\_uvm\_pyuvvm/module7/examples/best\_pra  
from pyuvvm import \*

ModuleNotFoundError: No module named 'pyuvvm'

# Practice & assessment

# Exercises

- DMA Verification
- Protocol VIP
- Best Practices
- Coverage Closure
- Final Project

# Assessment checklist

- Can verify complex designs
- Understands best practices
- Can create reusable VIP
- Can optimize performance
- Can achieve coverage closure
- Can plan verification projects

# Summary & next steps

- Apply UVM to real-world verification scenarios
- Complete module7/CHECKLIST.md
- Run `./scripts/module7.sh --check`
- Next: Next module in course